ΟΙΚΟΝΟΜΙΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ
Τμήμα Πληροφορικής
Μάθημα «Δίκτυα Υπολογιστών»
Εαρινό Εξάμηνο 2025-2026

Διδάσκων: Γεώργιος Δ. Σταμούλης, Καθηγητής
Υπεύθυνη Εργασίας: Δρ. Άννα Κεφάλα

Προγραμματιστική Εργασία Υλοποίησης Εφαρμογής: Κατανεμημένο Σύστημα Δημοπρασιών (Distributed Real-Time Auction System) – Φάση 1

Ημερομηνία Παράδοσης: 6/4/2026

1 Περιεχόμενο

Εξοικείωση με προγραμματισμό σε περιβάλλον υποδοχών (sockets) και με την υλοποίηση μηχανισμών ορισμένων πρωτοκόλλων.

2 Γενικές Πληροφορίες

Η εργασία θα πρέπει να εκπονηθεί σε ομάδες δύο (2) ή τριών (3) ατόμων. Συστήνεται να διατηρηθούν, κατά το δυνατόν, οι ίδιες ομάδες με τις άλλες εργασίες του μαθήματος. Τα παραδοτέα θα υποβληθούν μέσω του eclass ως αρχείο τύπου .zip ή .rar με όνομα της μορφής project1_AM1_AM2_AM3.zip, (όπου ΑΜ ο αριθμός μητρώου κάθε μέλους της ομάδας) την οποία θα υποβάλλει ηλεκτρονικά μέσω ***e-class*** ένα μέλος της ομάδας. Επίσης, θα ακολουθήσει και **προφορική εξέταση** της εργασίας στο εργαστήριο, που θα περιλαμβάνει σύντομη περιγραφή της εργασίας, των βημάτων που αναπτύχθηκαν, των δυσκολιών που αντιμετωπίσθηκαν κλπ.

3 Αντικείμενο της εργασίας

Η προγραμματιστική εργασία σε περιβάλλον υποδοχών αποτελεί εισαγωγή στο δικτυακό προγραμματισμό. Αντικείμενο της εργασίας είναι η υλοποίηση ενός απλού κατανεμημένου συστήματος διεξαγωγής online δημοπρασιών και συμμετοχής σε αυτές. Η εργασία θα υλοποιηθεί σε δύο φάσεις, με προοδευτικά αυξανόμενη πολυπλοκότητα. Η παρούσα εκφώνηση αναφέρεται στην Φάση 1.

Το σύστημα αποτελείται από έναν **Auction Server** και αρκετούς (τουλάχιστον 5) παρόμοιους **peers (clients) ή Bidders**, καθένας από τους οποίους διατηρεί ένα λογαριασμό στον Auction Server.

- **Auction Server**: διατηρεί τη λίστα με τα ενεργά αντικείμενα προς πώληση (όλων των πωλητών), τους τρέχοντες πλειοδότες και τις αντίστοιχες τιμές που προσφέρουν – σε κάθε χρονική στιγμή έχει μόνο μια δημοπρασία σε εξέλιξη.
- **Peers/Bidders** (clients): μπορούν να διαθέσουν ένα προς ένα αντικείμενα προς πώληση

(γίνονται πωλητές για κάθε συγκεκριμένο αντικείμενο) ή να δώσουν προσφορά “bid” για αντικείμενα άλλων τα οποία πωλούνται.

Κάθε peer έχει έναν **φάκελο** (shared_directory) στον οποίο τοποθετεί τα αντικείμενα που διαθέτει προς πώληση. Κάθε αντικείμενο περιγράφεται με ένα αρχείο που περιέχει συγκεκριμένα meta-data για το αντικείμενο. Ένας peer, αφού πρώτα συνδεθεί με τον Auction Server, τον ενημερώνει για κάθε νέο αντικείμενο που έχει διαθέσιμο προς πώληση και την επιθυμητή χρονική διάρκεια της δημοπρασίας. Ο Auction Server επιλέγει το αντικείμενο προς δημοπρασία. Όταν ένας peer επιθυμεί να συμμετάσχει σε δημοπρασία, συνδέεται στον Auction Server, βλέπει ποιο αντικείμενο δημοπρατείται τη δεδομένη στιγμή και ζητάει λεπτομέρειες (ποιος είναι ο πωλητής, ποια είναι η τρέχουσα μεγαλύτερη προσφορά (bid) και ποιος είναι ο υπολειπόμενος χρόνος για την ολοκλήρωση της δημοπρασίας αυτής). Εφόσον το αντικείμενο τον ενδιαφέρει και θέλει να κάνει προσφορά, στέλνει μήνυμα με την προσφορά στον Auction Server. Αν δεν υπάρξει νέα μεγαλύτερη προσφορά εντός του υπολειπόμενου χρόνου, η δημοπρασία λήγει και ενημερώνεται ο highest bidder ότι κέρδισε και ο πωλητής για το ποιος κέρδισε την δημοπρασία με ποια τιμή. Ο αγοραστής (highest bidder) επικοινωνεί κατευθείαν με τον πωλητή, του στέλνει μήνυμα ώστε να επιβεβαιώσει την αγορά και να παραλάβει το αρχείο με τα meta-data του αντικειμένου που αγόρασε.

Ο Auction Server καθώς και οι peers/bidders είναι σε θέση να διατηρούν **πολλαπλές συνδέσεις** με άλλους peers και να εξυπηρετούν τα αιτήματά τους ταυτόχρονα.

Η εργασία μπορεί να υλοποιηθεί σε γλώσσα python ή Java.

4 Λειτουργίες του Peer/Bidder

Κάθε peer εκτελεί τις εξής λειτουργίες:

1. Επικοινωνεί με τον Auction Server και τους άλλους peers με τη χρήση υποδοχών (sockets).
2. Διατηρεί τοπικά έναν φάκελο (**shared_directory**), στον οποίο αποθηκεύονται κάθε στιγμή τα αντικείμενα που θέλει να διαθέσει προς δημοπράτηση (ένα αρχείο με metadata για το αντικείμενο). Π.χ. Object_01.txt περιέχει [object_id: Object_01; description: “a description for specific object”; start_bid: “a number for value”; auction_duration: “time in sec”]. Σε κάθε συγκεκριμένο peer αυτά τα αντικείμενα (δηλ. τα σχετικά αρχεία) παράγονται σε τυχαία χρονικά διαστήματα, με χρήση διαδοχικών δειγμάτων της γεννήτριας τυχαίων αριθμών RAND ως εξής. Το πρώτο αντικείμενο παράγεται κατά την χρονική στιγμή RAND*120 sec. Το δεύτερο αντικείμενο παράγεται μετά από χρόνο ίσο με RAND*120 sec από την στιγμή που παράγεται το πρώτο κτλ, όπου χρησιμοποιείται νέο δείγμα της RAND.
3. Δημιουργεί και διατηρεί ένα λογαριασμό στον Auction Server (υλοποίηση λειτουργίας **register**, **login** και **logout**).
    a. **Register**: ο peer επιλέγει ένα username και password και τα στέλνει στον AuctionServer. Στην περίπτωση που το username δεν χρησιμοποιείται ήδη από άλλον peer, ο Auction Server δημιουργεί έναν λογαριασμό και επιστρέφει μήνυμα επιτυχίας

στον peer. Διαφορετικά επιστρέφει μήνυμα αποτυχίας δημιουργίας του λογαριασμού και απαίτησης για εισαγωγή νέου username από τον peer.

b. **Login**: ο peer εισάγει σε ένα μήνυμα το username και το password και το στέλνει στον Auction Server. Ο Auction Server πιστοποιεί τον peer και δημιουργεί ένα τυχαίο token_id με το οποίο και του απαντά. Το token_id είναι ένας τυχαίος αριθμός ο οποίος ταυτοποιεί μοναδικά τον peer για το συγκεκριμένο session (δηλαδή μέχρι ο peer να κάνει logout). Στη συνέχεια ο peer ενημερώνει τον Auction Server (λειτουργία **requestAuction**) για τα τρέχοντα περιεχόμενα του shared_directory του (αντικείμενα προς δημοπράτηση) καθώς και για τις πληροφορίες επικοινωνίας (ip_address, port).

c. **Logout**: ο peer ενημερώνει τον Auction Server ότι αποχωρεί (ταυτοποιώντας τον εαυτό του με το token_id). Ο Auction Server αποδέχεται την αποχώρηση, στέλνει μήνυμα επιτυχούς αποχώρησης στον peer, ενημερώνει κατάλληλα τις δομές δεδομένων που διατηρεί σχετικά με τους peers και ακυρώνει το token_id.

4. Υλοποιεί τη λειτουργία **requestAuction** με την οποία ενημερώνει τον Auction Server για τα αντικείμενά του προς δημοπράτηση (metadata) καθώς και για τα στοιχεία επικοινωνίας του (ip_address, port).
5. Υλοποιεί τη λειτουργία **getCurrentAuction**, με την οποία ζητά από τον Auction Server να τον ενημερώσει για το ποιο αντικείμενο δημοπρατείται (object_id και description) ανά τακτά διαστήματα, π.χ. ανά 1 λεπτό.
6. Υλοποιεί τη λειτουργία **getAuctionDetails**, με την οποία ζητά από τον Auction Server λεπτομέρειες για τη συγκεκριμένη δημοπρασία (εφόσον τον ενδιαφέρει να δώσει προσφορά), token_id του πωλητή, τρέχον highest bid και τον χρόνο που υπολείπεται για τη λήξη της δημοπρασίας. Η απόφαση για το αν ο peer ενδιαφέρεται για την συγκεκριμένη δημοπρασία λαμβάνεται με «στρίψιμο» νομίσματος (μέσω της γεννήτριας τυχαίων αριθμών RAND), με πιθανότητα επιτυχίας (δηλ. ενδιαφέροντος) ίση με 60%.
7. Υλοποιεί τη λειτουργία **placeBid**, με την οποία κάνει νέα μεγαλύτερη προσφορά για το αντικείμενο της δημοπρασίας (object_id, bid) στον Auction_Server, εφόσον φυσικά τον ενδιαφέρει αυτό. Ο υπολογισμός της νέας προσφοράς πρέπει να γίνεται βάσει του τύπου: NewBid=HighestBid*(1+RAND/10) οπότε η προσφορά αυξάνεται το πολύ κατά 10%.
8. Υλοποιεί τη λειτουργία **transaction**, με την οποία αν έχει ρόλο αγοραστή, συνδέεται με τον πωλητή (peer-to-peer) για να επιβεβαιώσει την αγορά του αντικειμένου object_id με τη συγκεκριμένη τιμή (bid). Αν έχει ρόλο πωλητή, στέλνει ως απάντηση το αρχείο με τα metadata του αντικειμένου και το σβήνει από το shared_directory του. Ο αγοραστής εφόσον λάβει το αρχείο με τα metadata, το αποθηκεύει στο shared_directory του και ενημερώνει τον Auction Server ότι έχει εκείνος πλέον το αντικείμενο.

5 Λειτουργίες του Auction Server

Ο Auction Server εκτελεί τις εξής λειτουργίες:

1. Διατηρεί κατάλληλη δομή δεδομένων με τις πληροφορίες κάθε εγγεγραμμένου peer (user_name, password, num_auctions-seller, num_auctions_bidder). Το num_auctions_seller είναι ένας μετρητής ο οποίος αυξάνεται +1 κάθε φορά που ο συγκεκριμένος peer συμμετέχει σε δημοπρασία ως πωλητής, ενώ το num_auctions_bidder είναι ένας μετρητής ο οποίος αυξάνεται +1 κάθε φορά που ο συγκεκριμένος peer κερδίζει τη δημοπρασία ως αγοραστής.
2. Υποστηρίζει τις λειτουργίες **register**, **login**, **logout** έτσι ώστε να εξυπηρετεί τις αντίστοιχες αιτήσεις των peers.
3. Διατηρεί κατάλληλη δομή δεδομένων που περιέχει τις πληροφορίες επικοινωνίας καθενός peer που είναι συνδεδεμένος αυτή τη στιγμή (token_id, ip_address, port, user_name) καθώς και τους αντίστοιχους μετρητές του num_auctions-seller και num_auctions_bidder.
4. Διατηρεί κατάλληλη δομή δεδομένων με τα metadata όλων των διαθέσιμων αντικειμένων προς δημοπράτηση κάθε peer (token_id, object_id, κλπ.). Η δομή δεδομένων αυτή χρησιμοποιείται για τη λειτουργία start_auction (βλέπε κατωτέρω). Στην αρχή είναι κενή και ενημερώνεται προοδευτικά καθώς συνδέονται peers με τον Auction Server και τον ενημερώνουν μέσω της λειτουργίας **requestAuction**.
5. Υποστηρίζει πολλαπλά νήματα ταυτόχρονα, δημιουργώντας ξεχωριστό νήμα για κάθε peer τον οποίο εξυπηρετεί.
6. Υλοποιεί τη λειτουργία **getAuctionRequest** με την οποία λαμβάνει τα αντικείμενα που κάθε peer-πωλητής έχει διαθέσιμα προς δημοπράτηση, τα τοποθετεί σε μία ουρά βάσει της σειράς σύνδεσης των peers που τα διαθέτουν (FCFS).
7. Υλοποιεί τη λειτουργία **startAuction** με την οποία πραγματοποιείται μια δημοπρασία την φορά. Συγκεκριμένα μόλις ολοκληρωθεί μια δημοπρασία, επιλέγει το πρώτο αντικείμενο στην ουρά από τα διαθέσιμα αντικείμενα προς δημοπράτηση. Αν η ουρά είναι άδεια, τότε ο Auction Server περιμένει την άφιξη του πρώτου αντικειμένου από κάποιον από τους peers. Ξεκινάει τη δημοπράτηση για το συγκεκριμένο αντικείμενο με αρχική τιμή (object_id, start_bid). Κάθε φορά που ένας υποψήφιος αγοραστής (peer/bidder) κάνει προσφορά (place_bid), ενημερώνει το current_bid και όλους τους ενεργούς peers για τη νέα προσφορά. Αν λήξει το χρονικό διάστημα χωρίς να δεχτεί νέα προσφορά, κατοχυρώνει το αντικείμενο στον highest bidder και στέλνει αντίστοιχο μήνυμα σε αγοραστή και πωλητή. Ενημερώνει κατάλληλα τις δομές δεδομένων και τους μετρητές num_auctions-seller, num_auctions_bidder. Για τον σκοπό αυτό πρέπει να χρησιμοποιηθούν σωστά κατάλληλοι μηχανισμοί κλειδώματος και κατάλληλες δομές δεδομένων (πχ. ConcurrentHashMap) ώστε να αποφευχθούν race conditions και deadlocks.
8. Υλοποιεί τη λειτουργία **sendCurrentAuction**, με την οποία απαντάει στον peer για την τρέχουσα δημοπρασία (object_id και description).
9. Υλοποιεί τη λειτουργία **sendAuctionDetails**, με την οποία απαντάει στον peer με λεπτομέρειες για τη συγκεκριμένη δημοπρασία token_id του πωλητή και τρέχον highest bid.

10. Υλοποιεί τη λειτουργία **checkActive** όταν ερωτάται από έναν peer μέσω της getCurrentAuction και getActionDetails (*ή/και σε οποιαδήποτε άλλη περίπτωση χρειαστεί να επιβεβαιώσει ότι είναι ενεργός κάποιος peer*). Ο Auction Server ελέγχει αν ο Seller είναι ακόμα online. Αν ο Seller αποσυνδεθεί ("πέσει" το socket), η δημοπρασία ακυρώνεται. Ενημερώνει κατάλληλα τις δομές δεδομένων και ακυρώνει το token_id του. Ενημερώνει όλους τους bidders που συμμετέχουν στη συγκεκριμένη δημοπρασία ότι ακυρώνεται η δημοπρασία.

Σημείωση: Σε όλες τις λειτουργίες πρέπει να εμφανίζονται κατάλληλα μηνύματα επιτυχίας ή αποτυχίας. Για ορισμένα από αυτά πρέπει να παρέχετε screenshots.

6 Υποδοχές (Sockets)

Ο Auction Server και οι peers/bidders θα επικοινωνούν μεταξύ τους μέσω sockets. Μια σύνδεση με sockets είναι ουσιαστικά μία σύνδεση δύο προγραμμάτων μέσω ενός δικτύου.

Στην συγκεκριμένη εφαρμογή:

- Ο Auction Server θα ακούει σε ένα Server Socket, το οποίο έχει συσχετιστεί με μία συγκεκριμένη θύρα.
- Κάθε peer ακούει και αυτός σε ένα Server Socket στο οποίο περιμένει είτε μηνύματα από άλλους peers είτε μηνύματα check_active από τον Auction Server.
- Με άλλα λόγια θα υπάρχουν δύο κατηγορίες συνδέσεων ανάλογα με το ποιος εκκινεί τη σύνδεση και με ποιον συνδέεται. Για τις λειτουργίες register, login, place_bid, κλπ. τη σύνδεση την εκκινεί ένας peer και συνδέεται με το Server Socket του Auction Server . Για τη λειτουργία transaction ο higher bidder peer εκκινεί τις συνδέσεις και συνδέεται με τα Server Socket του Seller peer.
- Η μόνη θύρα που πρέπει να θεωρείται εξ' αρχής γνωστή είναι αυτή στην οποία ακούει ο Auction Server . Όλες οι υπόλοιπες μπορούν να επιλέγονται τυχαία.

Για περισσότερες πληροφορίες σχετικά με τα sockets μπορείτε να ανατρέξετε στα:

https://docs.python.org/3/library/socket.html

http://docs.oracle.com/javase/tutorial/networking/sockets/index.html,

http://wiki.treck.com/Introduction_to_BSD_Sockets

7 Γενικές Πληροφορίες

Θα πρέπει να παραδώσετε σε ηλεκτρονική μορφή όλα τα αρχεία πηγαίου κώδικα (με επαρκή

σχολιασμό) της εφαρμογής και σχετικά βοηθητικά αρχεία. Επίσης θα πρέπει να παραδοθεί αναφορά που θα περιέχει:

1. Σύντομη ανάλυση της υλοποίησης.
2. Συνοπτική περιγραφή των αρχείων που αποτελούν την εφαρμογή, οδηγίες για το πως σχετίζονται, πως μεταγλωττίζονται και ποια είναι τα αρχεία εξόδου που προκύπτουν.
3. Τεκμηρίωση της εφαρμογής, π.χ. τρόπος εκτέλεσης προγραμμάτων, προβλήματα που αντιμετωπίσατε, αποκλίσεις από τις προδιαγραφές κτλ.
4. Screenshots της εξόδου που παράγει η εφαρμογή στις ακόλουθες περιπτώσεις:
   - Register ενός peer στον Auction Server (από την πλευρά του peer και του Auction Server).
   - Login ενός peer στον Auction Server (από την πλευρά του peer και του Auction Server).
   - Logout ενός peer στον Auction Server (από την πλευρά του peer και του Auction Server).
   - Πληροφορίες για την τρέχουσα δημοπρασία.
   - Επιτυχής προσφορά πλειοδοσίας.
   - Κατοχύρωση αντικειμένου στον higher bidder .
   - Επιτυχής ολοκλήρωση της συναλλαγής με αγορά του αντικειμένου (το αρχείο με τα metadata στο shared_directory του αγοραστή).
   - Ακύρωση δημοπρασίας λόγω αποσύνδεσης του πωλητή.
5. Άλλα screenshots που θεωρείτε ότι πρέπει να συμπεριλάβετε.